5. Graph Representation Using Adjacency List & BFS/DFS Traversal

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

class Graph {
private:
    int vertices;
    vector<vector<int>> adjList;

    void dfsHelper(int node, vector<bool>& visited) {
        visited[node] = true;
        cout << node << " ";
        for (int neighbor : adjList[node]) {
            if (!visited[neighbor]) dfsHelper(neighbor, visited);
        }
    }

public:
    Graph(int v) : vertices(v), adjList(v) {}

    void addEdge(int u, int v) {
        adjList[u].push_back(v);
        adjList[v].push_back(u);
    }

    void BFS(int start) {
        vector<bool> visited(vertices, false);
```

```
        queue<int> q;
        visited[start] = true;
        q.push(start);

        cout << "BFS starting from " << start << ": ";
        while (!q.empty()) {
            int curr = q.front();
            q.pop();
            cout << curr << " ";
            for (int neighbor : adjList[curr]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.push(neighbor);
                }
            }
        }
        cout << "\n";
    }

    void DFS(int start) {
        vector<bool> visited(vertices, false);
        cout << "DFS starting from " << start << ": ";
        dfsHelper(start, visited);
        cout << "\n";
    }
};

int main() {
    Graph g(5);
    g.addEdge(0, 1);
```

```
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(2, 4);

    g.BFS(0);
    g.DFS(0);
    return 0;
}
```